

Contents

1.1	Getting Started	2
1.1.1	Base Types	4
1.2	Classes and Objects	5
1.2.1	Creating and Using Objects	6
1.2.2	Defining a Class	9
1.3	Strings, Wrappers, Arrays, and Enum Types	17
1.4	Expressions	23
1.4.1	Literals	23
1.4.2	Operators	24
1.4.3	Type Conversions	28
1.5	Control Flow	30
1.5.1	The If and Switch Statements	30
1.5.2	Loops	33
1.5.3	Explicit Control-Flow Statements	37
1.6	Simple Input and Output	38
1.7	An Example Program	41
1.8	Packages and Imports	44
1.9	Software Development	46
1.9.1	Design	46
1.9.2	Pseudocode	48
1.9.3	Coding	49
1.9.4	Documentation and Style	50
1.9.5	Testing and Debugging	53
1.10	Exercises	55

1.1 Getting Started

Building data structures and algorithms requires that we communicate detailed instructions to a computer. An excellent way to perform such communication is using a high-level computer language, such as Java. In this chapter, we provide an overview of the Java programming language, and we continue this discussion in the next chapter, focusing on object-oriented design principles. We assume that readers are somewhat familiar with an existing high-level language, although not necessarily Java. This book does not provide a complete description of the Java language (there are numerous language references for that purpose), but it does introduce all aspects of the language that are used in code fragments later in this book.

We begin our Java primer with a program that prints “Hello Universe!” on the screen, which is shown in a dissected form in Figure 1.1.

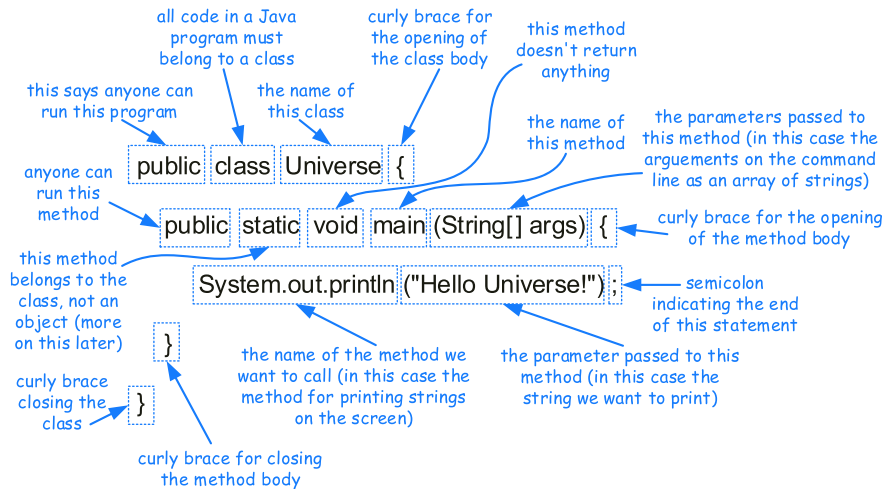


Figure 1.1: A “Hello Universe!” program.

In Java, executable statements are placed in functions, known as *methods*, that belong to *class* definitions. The Universe class, in our first example, is extremely simple; its only method is a static one named main, which is the first method to be executed when running a Java program. Any set of statements between the braces “{” and “}” define a program *block*. Notice that the entire Universe class definition is delimited by such braces, as is the body of the main method.

The name of a class, method, or variable in Java is called an *identifier*, which can be any string of characters as long as it begins with a letter and consists of letters, numbers, and underscore characters (where “letter” and “number” can be from any written language defined in the Unicode character set). We list the exceptions to this general rule for Java identifiers in Table 1.1.

Reserved Words				
abstract	default	goto	package	synchronized
assert	do	if	private	this
boolean	double	implements	protected	throw
break	else	import	public	throws
byte	enum	instanceof	return	transient
case	extends	int	short	true
catch	false	interface	static	try
char	final	long	strictfp	void
class	finally	native	super	volatile
const	float	new	switch	while
continue	for	null		

Table 1.1: A listing of the reserved words in Java. These names cannot be used as class, method, or variable names.

Comments

In addition to executable statements and declarations, Java allows a programmer to embed comments, which are annotations provided for human readers that are not processed by the Java compiler. Java allows two kinds of comments: inline comments and block comments. Java uses a “//” to begin an inline comment, ignoring everything subsequently on that line. For example:

```
// This is an inline comment.
```

We will intentionally color all comments in blue in this book, so that they are not confused with executable code.

While inline comments are limited to one line, Java allows multiline comments in the form of block comments. Java uses a “/*” to begin a block comment and a “*/” to close it. For example:

```
/*
 * This is a block comment.
 */
```

Block comments that begin with “/**” (note the second asterisk) have a special purpose, allowing a program, called Javadoc, to read these comments and automatically generate software documentation. We discuss the syntax and interpretation of Javadoc comments in Section 1.9.4.

1.1.1 Base Types

For the most commonly used data types, Java provides the following *base types* (also called *primitive types*):

boolean	a boolean value: true or false
char	16-bit Unicode character
byte	8-bit signed two's complement integer
short	16-bit signed two's complement integer
int	32-bit signed two's complement integer
long	64-bit signed two's complement integer
float	32-bit floating-point number (IEEE 754-1985)
double	64-bit floating-point number (IEEE 754-1985)

A variable having one of these types simply stores a value of that type. Integer constants, like 14 or 195, are of type **int**, unless followed immediately by an 'L' or 'l', in which case they are of type **long**. Floating-point constants, like 3.1416 or 6.022e23, are of type **double**, unless followed immediately by an 'F' or 'f', in which case they are of type **float**. Code Fragment 1.1 demonstrates the declaration, and initialization in some cases, of various base-type variables.

```

1 boolean flag = true;
2 boolean verbose, debug;           // two variables declared, but not yet initialized
3 char grade = 'A';
4 byte b = 12;
5 short s = 24;
6 int i, j, k = 257;                // three variables declared; only k initialized
7 long l = 890L;                    // note the use of "L" here
8 float pi = 3.1416F;               // note the use of "F" here
9 double e = 2.71828, a = 6.022e23; // both variables are initialized

```

Code Fragment 1.1: Declarations and initializations of several base-type variables.

Note that it is possible to declare (and initialize) multiple variables of the same type in a single statement, as done on lines 2, 6, and 9 of this example. In this code fragment, variables `verbose`, `debug`, `i`, and `j` remain uninitialized. Variables declared locally within a block of code must be initialized before they are first used.

A nice feature of Java is that when base-type variables are declared as instance variables of a class (see next section), Java ensures initial default values if not explicitly initialized. In particular, all numeric types are initialized to zero, a boolean is initialized to false, and a character is initialized to the null character by default.